End-to-End CI/CD Pipeline Documentation

SkillPulse Automated Containerization & GitOps Deployment

1. Architectural Flow Overview

This pipeline automates the journey of your application code from a local developer environment to a production-ready Kubernetes cluster using **GitHub Actions**, **Docker Hub**, and **GitOps** principles:

1. **Local Commit & Push:** The developer pushes clean code changes to the `main` branch of the repository.
2. **GitHub Actions Trigger:** The GitHub Actions runner spawns a secure virtual environment (`ubuntu-latest`).
3. **Secure Authentication:** The runner securely injects credentials (`DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`) directly from your GitHub Repository Secrets.
4. **CI Stage (Build & Push):**
   - The repository workspace is checked out.
   - Docker Buildx engine is initialized.
   - Docker builds distinct, production-optimized images for both `/backend` and `/frontend` using their respective `Dockerfile`s.
   - The images are tagged with `latest` and unique `SHA` hashes, then pushed securely to Docker Hub.
5. **CD Stage (GitOps / Cluster Update):**
   - The successful build triggers the Continuous Delivery (CD) workflow.
   - It updates (bumps) the image tags inside your Kubernetes (`k8s`) manifest files to point to the newly built image.
   - The **Kind Cluster** pulls the fresh images from Docker Hub and updates the running containers seamlessly.

2. Complete Workflow File (`.github/workflows/ci.yml`)

```
name: CI Workflow

on:
  push:
    branches: [ main ]
    paths-ignore:
      - 'k8s/**'
      - 'docs/**'
      - 'README.md'

jobs:
  build-and-push:
    runs-on: ubuntu-latest

    steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    - name: Set up Docker Buildx
```

```
      uses: docker/setup-buildx-action@v3

    - name: Login to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.DOCKERHUB_USERNAME }}
        password: ${{ secrets.DOCKERHUB_TOKEN }}

    - name: Build and Push Backend
      uses: docker/build-push-action@v5
      with:
        context: ./backend
        push: true
        tags: |
          ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-backend:latest
          ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-backend:${{ github.sha }}

    - name: Build and Push Frontend
      uses: docker/build-push-action@v5
      with:
        context: ./frontend
        push: true
        tags: |
          ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-frontend:latest
          ${{ secrets.DOCKERHUB_USERNAME }}/skillpulse-frontend:${{ github.sha }}
```

3. Terminal Commands Used & Detailed Explanations

A. Git Integration Commands

`git add .github/workflows/ci.yml`

Purpose: Stages the workflow configuration file. It instructs Git to start tracking changes in this file so they are prepared to be packaged into the next commit.

`git commit -m "Fix docker login secrets and typo"`

Purpose: Permanently saves the staged changes to your local Git history. The `-m` flag attaches a clear, descriptive message, enabling easy change tracking (e.g., resolving the `s crets` syntax error).

`git push origin main`

Purpose: Uploads your local commits to the remote GitHub repository under the `main` branch. This action triggers the GitHub Actions workflow runner automatically.

B. Linux Command-Line Editing

`nano .github/workflows/ci.yml`

Purpose: Opens the text file inside the terminal-based Nano editor to let you modify code and configurations directly on the command line.

`Ctrl + O` followed by `Enter`

Purpose: Saves (writes out) your current edits to the disk without closing the editor.

`Ctrl + X`

Purpose: Closes the Nano editor and safely returns you to your standard shell terminal.

4. Core Troubleshooting & Best Practices

- **Secrets Case-Sensitivity:** Ensure labels in GitHub Settings under *Secrets and variables > Actions* perfectly match your YAML configuration (e.g., `DOCKERHUB_USERNAME` must match exactly).
- **Strict YAML Whitespace:** YAML files rely on space indentation. Be extremely careful with templates such as `${{ secrets.NAME }}` . Typographical spaces (like `s crets`) will break parsing completely.
- **Folder Context Mismatch:** Always verify that paths like `./backend` or `./frontend` contain a valid `Dockerfile` at their root levels.